PROTOCOLO VERMILHÃO — OPERAÇÃO ÚLTIMA ENERGIA

Relatório técnico — Trabalho Prático 1 de Computação Gráfica

IFSULDEMINAS, Campus Machado · Prof. Renan · Entrega: 11/11/2026 · Integrantes: Gabrieli Diniz e [nome da dupla]

1. Descrição da proposta

Minijogo 3D em Python com Pygame (janela, teclado, fonte) e PyOpenGL na API clássica, sem shaders. O jogador pilota um drone de manutenção por uma estação espacial abandonada, coleta 5 núcleos de energia, contorna 19 obstáculos (caixas, contêineres, pilares e uma torre central) e entra no portal de saída quando ele é reativado. Todos os modelos são construídos por código com primitivas, em estilo low poly.

Identidade visual. Vermelho #E63946 (núcleos, alertas, indicadores, portal), vermelho escuro #8B1E2D, preto azulado #10141D (fundo), cinza metálico #596273 (estruturas), cinza claro #D8DEE9 (textos) e branco (informações importantes). **Diferencial: a iluminação faz parte da história.** A estação começa escura, com luzes de emergência vermelhas; a cada núcleo coletado a energia é restaurada — painéis acendem, a luz geral sobe e, em 5/5, o portal se abre. A tecla L liga/desliga a luz principal em qualquer etapa.

2. Capturas de tela

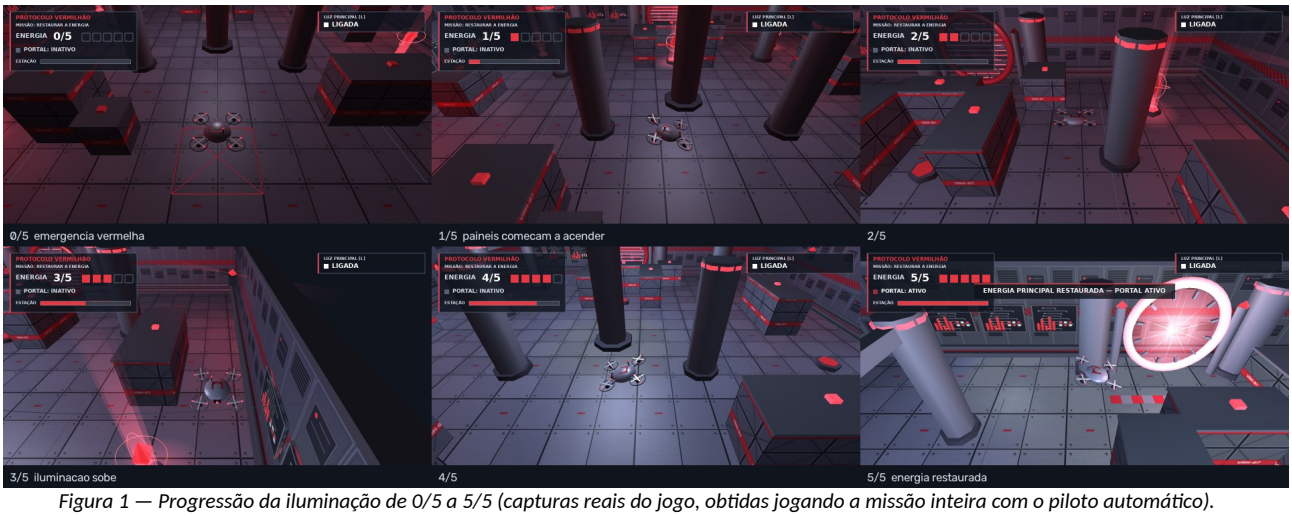


Figura 1 — Progressão da iluminação de 0/5 a 5/5 (capturas reais do jogo, obtidas jogando a missão inteira com o piloto automático).

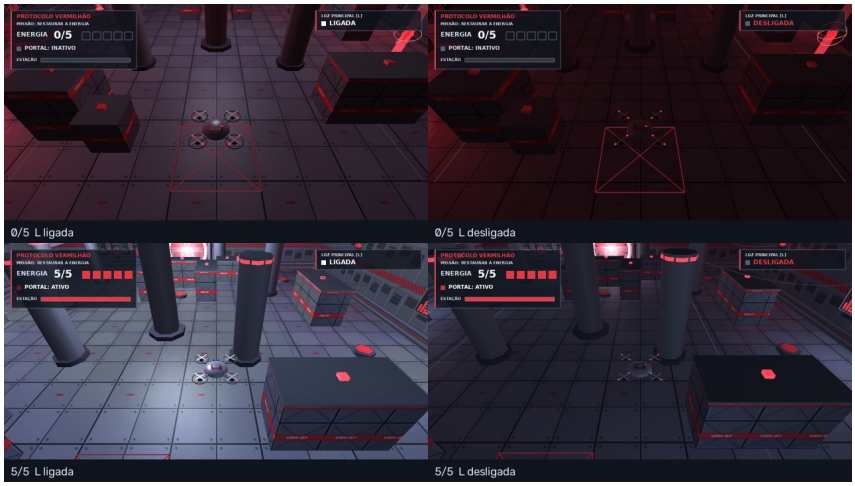


Figura 2 — Tecla L: luz principal ligada × desligada em 0/5 (alto) e em 5/5 no centro da estação (baixo).

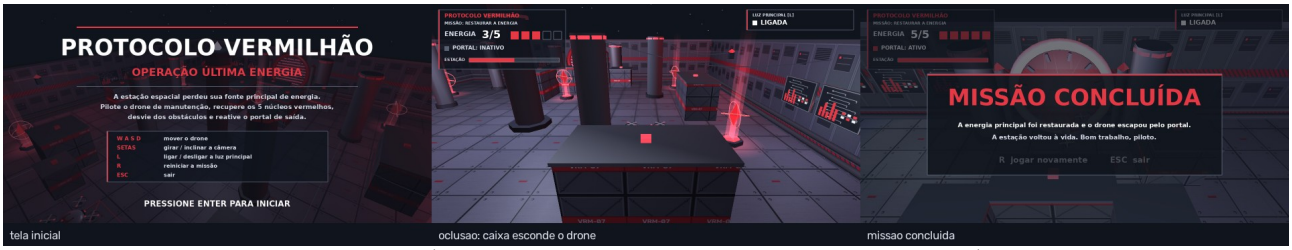


Figura 3 — Tela inicial; oclusão (o contêiner esconde o drone: o teste de profundidade funciona); tela de vitória.

3. Requisitos de computação gráfica e sua implementação

Técnica exigida	Aplicação no jogo	Onde
Primitivas	GL_QUADS (caixas, piso, paredes, painéis); GL_TRIANGLES (cristal = octaedro de 8 triângulos); GL_TRIANGLE_FAN / GL_QUAD_STRIP (discos, esfera, toros, prismas); GL_LINES / GL_LINE_LOOP (cabos, marcações do piso, anéis orbitais); GL_POINTS (estrelas).	primitivas.py, objetos.py
Drone composto	Corpo (esfera achatada), base, antena, 4 braços, 4 motores, 4 anéis (toro), 8 lâminas, sensor	Cenario.desenhar_

	frontal com lente vermelha e LEDs vermelhos.	drone
Coordenadas, câmera e projeção	Vértices em coordenadas locais → mundo (glTranslate/Rotate/Scale) → câmera 3ª pessoa elevada (gluLookAt, segue o drone com atraso suave) → projeção em perspectiva (gluPerspective 60°) → pixels. GL_DEPTH_TEST resolve a oclusão.	cena.py
Translação	Posição do drone, dos núcleos, de cada obstáculo e do portal; flutuação dos núcleos.	estado.py, objetos.py
Rotação	Orientação (yaw) e inclinação do drone; giro das hélices, dos núcleos, do giroflex e do aro do portal.	objetos.py
Escala	Cubo unitário escalado em cada peça do drone; viga do pórtico; núcleo encolhendo ao ser coletado; esfera/octaedro achatados ou esticados.	objetos.py
Transf. hierárquica	Hélice dentro do braço, dentro do drone (seção 4).	desenhar_drone
Animações	Contínua: hélices e núcleos. Periódica: núcleos flutuam (seno), drone oscila, luzes de emergência pulsam. Por evento: núcleo encolhe e some em 0,5 s; portal abre em 2,6 s; LED do drone pisca ao colidir; iluminação restaura em transição suave.	estado.py, iluminacao.py
Texturas	4 texturas procedurais 256×256: piso metálico, parede com painéis, caixa de manutenção, painel de energia. Coordenadas de textura em todas as superfícies; piso repete 10×10 (GL_REPEAT); nas caixas as coordenadas são proporcionais ao tamanho (sem esticar).	texturas.py, gerar_texturas.py
Iluminação	Ambiente global + 8 luzes pontuais (difusa, especular, atenuação), normais por face, materiais (GL_COLOR_MATERIAL, GL_SPECULAR, GL_SHININESS, GL_EMISSION). Brilho especular no corpo do drone, que muda com a orientação.	cena.py, iluminacao.py
Interação e regras	WASD (relativo à câmera), limites da estação, colisão círculo × caixa com deslizamento nos 19 obstáculos, coleta única por núcleo, contador no HUD e no título, tela de vitória, reinício com R sem fechar.	estado.py, main.py, hud.py

4. Transformação hierárquica: a hélice

Cada hélice é "filha" do braço, que é "filho" do drone. Os glPushMatrix/glPopMatrix aninhados fazem a matriz de cada nível herdar a do pai:

```
glPushMatrix()                                  # DRONE
  glTranslatef(d.x, d.y, d.z); glRotatef(d.yaw, 0,1,0); glRotatef(-d.inclinacao, 1,0,0);
glScalef(.72,.72,.72)
  for k in range(4):
    glPushMatrix(); glRotatef(45+90*k, 0,1,0); glTranslatef(1.5, 0, 0)      # BRAÇO k (aponta para fora)
      glPushMatrix(); glTranslatef(0, .24, 0); glRotatef(±d.angulo_helice, 0,1,0)   # HÉLICE: giro local
        desenha as lâminas
      glPopMatrix()
    glPopMatrix()
glPopMatrix()
```

Matriz final da hélice = T(drone)·R(yaw)·R(inclinação)·S·R(braço)·T(ponta)·T(altura)·R(giro). Só o último fator é a animação própria; todos os anteriores vêm do pai. Por isso a hélice gira em torno do próprio centro e, ao mesmo tempo, acompanha a posição, a orientação e a inclinação do drone. O mesmo padrão aparece no aro dentado que gira dentro do anel do portal.

5. Uso de dt (tempo entre quadros)

A cada quadro, main.py mede dt = relogio.tick(60)/1000 (segundos; limitado a 0,05 s). Toda grandeza que varia no tempo é multiplicada por dt: posição += velocidade·dt; ângulo += graus_por_segundo·dt; escala do núcleo = f(tempo_de_coleta/0,5 s); abertura do portal += dt/2,6. Suavizações (aceleração do drone, câmera e transição da luz) usam k = 1 − e^(−taxa·dt), que dá o mesmo resultado qualquer que seja o FPS. Verificação: simulando 1,5 s de movimento com dt = 1/30, 1/60 e 1/144 o drone termina em z = 2,76, 2,82 e 2,85 (diferença < 0,1 m, efeito da integração discreta); a transição de luz difere menos de 0,005 entre 30 e 120 FPS.

6. Texturas, normais e iluminação

Texturas: geradas por gerar_texturas.py (NumPy + OpenCV), autoria própria — nenhuma imagem externa. Os arquivos são lidos por caminho relativo a __file__ (e recriados por código se faltarem). Normais: cada face de caixa/prisma/octaedro tem normal própria (aparência low poly); a esfera do drone usa normais suaves, e GL_NORMALIZE corrige as normais sob glScalef. Piso e paredes são malhas de 1 m: como a OpenGL clássica ilumina por vértice, a malha fina faz as "poças" de luz aparecerem. GL_LIGHT_MODEL_LOCAL_VIEWER e GL_SEPARATE_SPECULAR_COLOR deixam o brilho especular correto e visível também sobre texturas.

Luz	Função	Comportamento
LIGHT0	Luz principal (torre central) — tecla L	Branco-azulada; intensidade 0,62 (0/5) → 1,25 (5/5); fade de 0,2 s ao ligar/desligar; independe da missão.
LIGHT1–2	Emergência (paredes)	Vermelhas e pulsantes; 100% em 0/5 → 30% em 5/5; giroflex rotativo.
LIGHT3	Portal	Vermelho fraco enquanto fechado; branco-avermelhado intenso conforme abre.
LIGHT4–7	4 grupos de painéis	Acendem em 1/5, 2/5, 3/5 e 4/5 (55%) e vão a 100% em 5/5; painéis também ganham emissão.
Ambiente	GL_LIGHT_MODEL_AMBIENT	Vermelho escuro (0,17; 0,10; 0,11) → azul-acinzentado (0,17; 0,19; 0,23): nunca escuro demais para jogar.

O nível de restauração segue a tabela 0 → 0,12 → 0,25 → 0,50 → 0,75 → 1,0 (0/5 … 5/5), aproximado de forma exponencial com dt. Como são luzes reais, a diferença vem da interação luz × normal × material, e não de trocar cores de objetos (Figuras 1 e 2). Os LEDs, painéis e lâmpadas usam GL_EMISSION apenas como acabamento visual; quem ilumina o ambiente são as fontes.

7. Testes, limitações e referências

Testes executados: 21 testes automáticos de lógica (limites, colisão contra cada obstáculo, coleta única, animação de coleta, portal, reinício, independência de FPS, progressão e independência da tecla L) e 5 de integração (janela real: sem erro de OpenGL em menu/jogo/portal/vitória, teclas ENTER/L/R/ESC, hélices mudando a imagem, luz L alterando os pixels). Um piloto automático completou a missão pelo código do jogo (5 núcleos e portal em 27,5 s simulados). As capturas foram geradas por testes/capturar_telas.py. **Observação:** os testes visuais foram executados com renderização por software (Mesa), portanto o FPS em placa de vídeo não foi medido (~32 ms/quadro nesse ambiente).

Limitações: altura do drone constante; colisão 2D (círculo × caixa); iluminação por vértice (sem sombras projetadas; pode haver leve aspecto facetado nas poças de luz); máximo de 8 luzes da OpenGL clássica; câmera sem colisão com objetos; sem som; texturas procedurais simples.

Referências: notebooks das atividades de OpenGL e transformações da disciplina; documentação do Pygame (pygame.org/docs) e do PyOpenGL (pyopengl.sourceforge.net); Khronos OpenGL Wiki — Fixed Function Pipeline (iluminação e materiais); Shreiner et al., OpenGL Programming Guide (Red Book). Texturas e modelos: autoria própria, gerados por código.